

Notebook

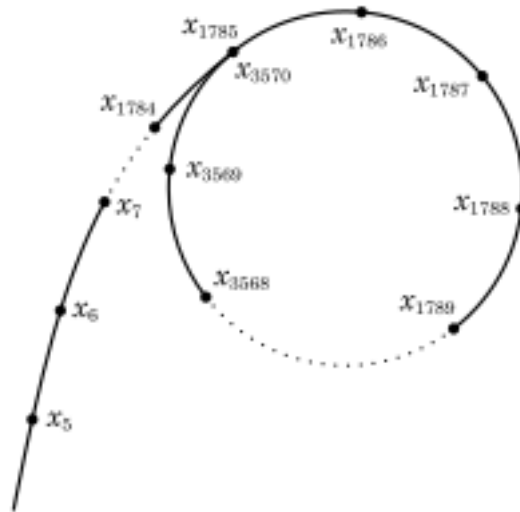
February 8, 2026

1 Pollard's rho and lambda (kangaroo) algorithms for Discrete Logarithm

We've learnt how to solve Discrete Logarithm Problem for multiplicative groups with order $p - 1$ with small factors. What if p is a safe prime? It would take you a few days to bruteforce all possible solutions to DLP with p consisting of 40 bits. But there is actually a better way.

Let's revise our basics: we have a safe prime p , a generator g , a private key a and a public key $A = g^a \bmod p$.

Pollard's rho algorithm is a probabilistic algorithm, which allows to solve DLP in time $O(\sqrt{p})$. It is named ρ because of the shape of the search space, which you can see in this image, shamelessly taken from wikipedia.



The basic idea behind the algorithm is quite simple even though it has a funny form. We create two “kangaroos” (one Tame and one Wild) which jump through the space of the multiplicative group. These jumps are guided by a deterministic function f . For example, you can use the following function (but you might need to update it for the actual task):

```
[ ]: def f(x,m):
      return 1<<(x%m)
```

Where m is decided by the size of the group $p - 1$. From my experience $m = \frac{\log_2(p-1)}{2}$ works quite well.

So one jump is:

$$x_n = x_{n-1} + f(y_{n-1}), y_n = y_{n-1} \cdot g^{f(y_{n-1})} \bmod p, \text{ so } y_n = y_0 * g^{x_n}$$

The idea behind these jumps is that if two kangaroos land on the same element once, they will continue to land on the same elements, since the $f(x)$ function is deterministic.

Back to our kangaroos.

The Tame one starts from the end:

$yT = g^d \bmod p, xT = d$ (notice that we aren't computing xT in the field modulo p . Our goal here is to track the logarithm **and** the number of steps that we've taken)

and starts jumping ('^' denotes exponentiation):

```
for i in {0,k}
do
    xT = xT + f(yT)
    yT = yT * (g ^ f(yT)) % p
done
```

You can try out different k , a good choice is the average of all possible values generated by function f multiplied by 4.

The idea is that the Tame kangaroo jumps through the multiplicative group circling it several times and we know the discrete logarithm of the element it lands on.

Now we start jumping with the Wild kangaroo:

```
xW = 0
yW = A
while xW <= xT
do
    xW = xW + f(yW)
    yW = yW * (g ^ f(yW)) % p
    if yW = yT
    then
        stop
    fi
done
```

If we just run out of xT we need to change parameters and start anew.

The algorithm is probabilistic, it is not guaranteed to find the logarithm on the first try. However, when you restart the algorithm, you need to change the hashing function

$f(x)$, since this one failed. For example, you can use $1 \ll ((x*k+l)\%m)$, where k and l are picked randomly at the start of one try, but be sure not to set k to zero.

If we find an $yW = yT$, we can learn a :

$$a = (xT - xW) \bmod p - 1$$

The idea is that the Wild kangaroo jumps through the elements of the multiplicative group based off of the element we want to get the logarithm for. If at some point it lands on one of the elements that the Tame kangaroo has previously landed on, it is doomed to jump the same path from then on. By checking the added exponent xW we are making sure that the Wild kangaroo has taken enough jumps to catch up to the Tame one. And if they land on the same element, then from the difference in jumps they have taken we can easily recover the logarithm of the starting element.

Now try it yourself locally with small parameters first:

```
[ ]: import random
p=882719
a=random.randint(2,p-2)
g=2
A=pow(g,a,p)
def pollard_lambda(A,g,p):
    #Your code goes here
    return 1
if pollard_lambda(A,g,p)==a:
    print ("Success")
else:
    print ("Fail")
```

Let's look at another case. What if p is 40 bits, but $a = k*b + c$, where k is 20 bits long, and k and c are known?

This case is also quite easy to crack, but it uses a slightly different version of the algorithm, called Pollard's lambda algorithm (once again, it's because of how the letter λ looks). Now the goal is not to run around the multiplicative group, but rather to chase after each other on a straight line. The lambda version works for the case when the exponent is in the defined interval ($start, finish$). In this case $m = \frac{\log_2(finish-start)}{2}$.

The idea is that the Tame kangaroo starts at the finish, then uses the same algorithm as ρ but by changing the parameter m it doesn't jump through the whole group, but just through some runway after the interval (defined by parameter k and supposed to be larger than the interval). Then the Wild kangaroo starts jumping. Since it was somewhere within the interval originally, we hope that it will land on the same element as the Tame one somewhere on the runway and will also finally land on the same element we finished on with the Tame one.

Now back to the specific case we were discussing. You start your Tame Kangaroo with $yT = g^{k*l+c}$, where l is the highest possible number such that $k*l + c < p - 1$ and use g^k as the new generator.

Then you do the Wild kangaroo with the new generator, and m with $\lceil \frac{p}{k} \rceil$. This will

allow you to compute b and therefore a .

Try it:

```
[ ]: p=1088364193559
k=891239
b=random.randint(1,(p//k)-1)
a=b*k
g=2
A=pow(g,b*k,p)
def pollard_lambda_mult(A,g,p,k):
    #Your code goes here
    return 1
if pollard_lambda_mult(A,g,p,k)==a:
    print ("Success")
else:
    print ("Fail")
```

Now that you've tested the functions locally, let's try it on our challenge. You will need to solve 2 problems and will get 2 flags.

```
[ ]: import socket
import re
class VulnServerClient:
    def __init__(self, show=True):
        """Initialization, connecting to server"""
        self.s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
        self.s.connect(('cryptotraining.zone',1346))

    def recv_until(self, symb=b'\n>'):
        """Receive messages from server, by default till new prompt"""
        data=b''
        while True:

            data+=self.s.recv(1)
            if data[-len(symb):]==symb:
                break
        return data
    def getChallenge(self, show=True):
        data=self.recv_until()
        try:
            data=data.decode()
        except UnicodeDecodeError:
            print ('Error decoding unicode. Try connecting to server_
↵again.')
            return (None, None)
        if show:
            print (data)
```

```

p=int(re.search(r'(?<=p=)\d+',data).group(0))
A=int(re.search(r'(?<=2\*\*k \(\b\b p\)=)\d+',data).group(0))
return (p,A)

def checkSolution(self,k, show=True):
    """Submit solution"""
    self.s.sendall((str(k)+'\n').encode())
    data=self.recv_until()
    try:
        data=data.decode()
    except UnicodeDecodeError:
        print ('Error decoding unicode. Try connecting to server_
again.')
```

```

        return None
    if show:
        print (data)
    if data.find('flag')!=-1:
        return True
    return False

def checkSolution1(self,k,show=True):
    """Submit first solution"""
    return self.checkSolution(k,show)

def checkSolution2(self,k,show=True):
    """Submit second solution"""
    return self.checkSolution(k,show)
def __del__(self):
    self.s.close()

```

```

[ ]: vs=VulnServerClient()
g=2
(p,A)=vs.getChallenge()
print(p, A)

```

```

[ ]: solution1=1# put your solution to the first problem here
vs.checkSolution1(solution1)

```

```

[ ]: (p,A)=vs.getChallenge()
print(p, A)
#k=10000 (mod 832661519819)
k1=10000
p1=832661519819
solution2 =1 # put your solution to the second problem here
vs.checkSolution2(solution2)

```